DIGITAL EGYPT PIONEERS INITIATIVE (DEPI)

Round 4 (2025–2026)

GRADUATION PROJECT REPORT

Satellite Mission Operations Monitoring System Using Skyfield, Delta Lake, and Medallion Architecture

Course Instructor: Eng. Mohamed Hamed

Prepared By:

- Jana Mohamed Hassan Habachy (Project Lead & Architecture Design)
- Hanin Galal (Data Processing & Visualization)
- Malak Assal (Automation & Pipeline Orchestration)

Submission Date: July 2026

Acknowledgements

The execution and successful completion of this engineering graduation capstone project were made possible through the support, guidance, and institutional framework provided by the Digital Egypt Pioneers Initiative (DEPI). We express our deep appreciation to our program instructor, Eng. Mohamed Hamed, whose technical expertise, structured feedback, and rigorous evaluation of our pipeline architecture continuously pushed our team to align our implementation with production-grade data engineering standards.

We also acknowledge the academic and logistical resources provided by the DEPI management team, which allowed us to explore cloud-native analytics architectures, collaborative development environments, and complex data pipeline orchestrations.

Finally, this project stands as a testament to intensive collaborative effort. Each team member contributed dedicated technical competencies—ranging from complex orbital mathematics to robust PySpark schema structures and analytical visual interfaces. The synergy within our team allowed us to troubleshoot architectural roadblocks, pivot efficiently under platform constraints,

and deliver a comprehensive software and data architecture that demonstrates the power of modern data pipelines.

\newpage

Abstract

Modern space systems and satellite operations require high-throughput, low-latency telemetry processing frameworks to guarantee mission safety, monitor orbital parameters, and extract operational insights. Traditional telemetry architectures frequently suffer from rigid structural schemas, a lack of data lineage, and limited horizontal scalability. This graduation capstone project addresses these shortcomings by engineering a fully automated, end-to-end Satellite Mission Operations Monitoring System built entirely on a three-tier Medallion Architecture using Delta Lake storage within Azure Databricks.

Instead of relying on static, pre-existing historical datasets that fail to capture the dynamic nuances of orbital pathways, real-time tracking data is generated synthetically. This is achieved by utilizing the Skyfield astronomical library to parse two-line element (TLE) sets and model propagation vectors relative to designated terrestrial ground stations. The raw streaming ingestion framework writes high-frequency state vectors directly into an immutable Delta Lake Bronze tier.

A programmatic validation and quality enforcement engine then systematically filters anomalies, applies strict physical boundary constraints, and handles deduplication routines to transition clean data into the Silver tier. Aggregation metrics and windowing queries evaluate telemetry windows to compute satellite communication pass analytics, compute link budget KPIs, and drive an automated multi-threshold alert state engine inside the Gold tier.

Due to structural license limitations within the cloud platform's free tier, the live cloud-to-BI link was restricted. The final Gold data layers were exported as structured, schema-validated CSV payloads and ingested manually into Power BI Desktop to build a comprehensive, multi-page mission control dashboard. This report details the complete software engineering life cycle, design trade-offs, configuration frameworks, and testing criteria that validate this resilient architecture for modern space mission analytics.

\newpage

Table of Contents

List of Figures

List of Tables

\newpage

Glossary

- **AOS**: Acquisition of Signal. The specific timestamp when a satellite rises above a ground station's local horizon and establishes a viable communications link.
- **APA**: American Psychological Association. A standard formatting and citation style utilized for technical and academic reports.
- **API**: Application Programming Interface. A set of defined protocols and tools that allows different software applications to communicate with one another.
- **Azimuth**: The horizontal angular direction of a celestial object or satellite, measured clockwise around the observer's horizon from true north (0° to 360°).
- **BI**: Business Intelligence. Technology architectures and visualization frameworks used to analyze data and present actionable insights.
- **Bronze Table**: The initial storage tier within a Medallion Architecture where raw, unaltered data streams are appended directly from source systems.

- **CI/CD**: Continuous Integration and Continuous Deployment. Automated software engineering pipelines designed to test and deploy code changes.
- **CSV**: Comma-Separated Values. A common, plain-text file format used to store tabular data structures.
- **DBFS**: Databricks File System. A virtual, cluster-mounted file system interface that abstracts underlying cloud object storage.
- **DEPI**: Digital Egypt Pioneers Initiative. The national software and data engineering talent development program framing this project.
- **Delta Lake**: An open-source storage layer that brings ACID transactions, data versioning, and scalable metadata management to big data files.
- **Elevation**: The vertical angular height of a satellite measured from the observer's local horizontal plane (0° to 90°).
- **ETL**: Extract, Transform, Load. A primary data engineering workflow pattern responsible for transferring and restructuring data across systems.
- **Gold Table**: The final analytical tier within a Medallion Architecture, structured for rapid reporting, operational KPIs, and presentation.
- **Hz**: Hertz. The fundamental unit of frequency, representing cycles per second.
- **ISS**: International Space Station. The orbital satellite platform utilized as our baseline TLE tracking target during pipeline execution.
- **JDBC**: Java Database Connectivity. An industry-standard database abstraction layer utilized to connect BI clients to remote execution clusters.
- **KPI**: Key Performance Indicator. A quantifiable metric used to evaluate operational success and performance levels.
- **LEO**: Low Earth Orbit. An orbital regime stretching from Earth's surface up to approximately 2,000 kilometers, characterized by rapid orbital periods.
- **LOS**: Loss of Signal. The specific timestamp when a satellite drops below a ground station's local horizon, terminating the active communication pass.
- **Medallion Architecture**: A data design pattern consisting of three distinct processing layers (Bronze, Silver, Gold) to systematically clean and enrich data.
- **ODBC**: Open Database Connectivity. A standard API interface utilized to bridge visualization desktop applications with cloud warehouses.
- **PAT**: Personal Access Token. A secure, user-generated string used for API and application authentication across cloud networks.
- **PySpark**: The Python API language framework designed to interface with Apache Spark's distributed big data processing core.
- **SAS**: Shared Access Signature. An access token architecture used to manage read/write permissions securely on cloud storage objects.
- **Silver Table**: The intermediate storage tier within a Medallion Architecture where data is cleansed, type-cast, enriched, and deduplicated.
- **Skyfield**: A high-precision Python astronomical library utilized to calculate orbital mechanics and propagate positions using standard TLE strings.
- **TLE**: Two-Line Element. A standardized data format containing a list of orbital perturbations and parameters used to track satellites relative to a specific epoch.

- **VM**: Virtual Machine. A software-defined computer system running on cloud server infrastructure.

\newpage

1. Introduction

Space operations and satellite communications networks generate vast, continuous arrays of tracking telemetry. This data must be actively parsed, cleaned, analyzed, and visualized to ensure system integrity, maintain communications links, and prevent catastrophic failures. In high-stakes satellite operations, processing lag, data dropouts, or corrupted state logs can lead to lost contact windows or delayed anomaly responses.

As low Earth orbit (LEO) constellations rapidly expand, space ground stations require automated, scalable, and resilient data processing architectures capable of transforming raw tracking metrics into operational business intelligence in near-real-time.

Historically, satellite mission control centers relied on siloed, proprietary client-server infrastructure to monitor live health and state vectors. These legacy frameworks often process telemetry streams using custom software stacks that lack the modularity, horizontal scalability, and standard analytical integration capabilities found in modern enterprise big data platforms.

When long-term historical trends or aggregated orbital analytics are required—such as calculating link budget performance, evaluating structural signal degradation patterns across specific geographic axes, or auditing alert severity frequencies—it becomes difficult to extract this information from raw binary telemetry files or transactional relational databases.

This capstone project implements a modern, cloud-native telemetry processing pipeline. It bridges the gap between scientific orbital mechanics and production-grade data engineering by building an end-to-end Satellite Mission Operations Monitoring System within the Azure Databricks platform.

The system leverages a multi-tier Medallion Architecture backed by Delta Lake storage. Instead of utilizing static, pre-existing historical CSV files that cannot adapt to layout modifications or testing scenario updates, the pipeline programmatically generates highly precise satellite state models. It maps standard North American Aerospace Defense Command (NORAD) Two-Line Element (TLE) structures using the Skyfield astronomical library. This library handles position propagation relative to specialized terrestrial coordinates.

The system automates the ingestion of raw telemetry streams into an immutable, transactionally secure Bronze storage layer. From there, it cleanses and type-casts data into a validated Silver layer, computes structural summaries within a Gold analytics tier, and visualizes the results via an intuitive multi-page Power BI dashboard dashboard.

The sections that follow document our engineering logic, framework selection, algorithm design, verification models, and operational results. This showcases a resilient design that aligns directly with modern enterprise data intelligence strategies.

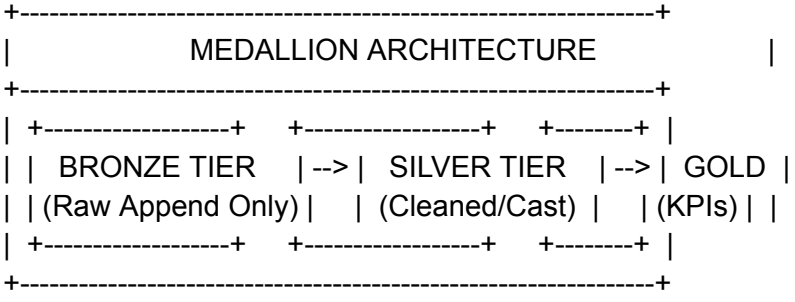
2. Literature Review

The development of scalable data infrastructure for telemetry processing intersects two primary areas of computer science research: distributed cloud storage architectures and automated astronomical state simulation frameworks. To understand the design decisions of this system, we must evaluate the historical progression from flat-file architectures to modern transactional data lakes, alongside the operational advantages of procedural orbital modeling over legacy static data paradigms.

For decades, large-scale telemetry frameworks relied on structured text layouts (such as CSV or JSON) or specialized binary formats (like HDF5 or NetCDF) written directly to network-attached disks or distributed file structures. While these layouts are highly effective for simple archiving, they struggle when integrated into high-frequency downstream analytical applications.

Flat files lack metadata catalog rules, cannot enforce schema changes natively, and do not provide atomic guarantee transactions. If an injection job fails midway through a write cycle, the file system is left in a corrupted state. This requires manual data cleanup and results in missing structural context. Furthermore, running multi-variable SQL aggregations or horizontal lookup trends over millions of raw historical files requires reading full directories sequentially. This creates major bottleneck constraints on network IO.

The introduction of the Medallion Architecture—pioneered within lakehouse designs using storage engines like Delta Lake—fundamentally changed how enterprise telemetry data is structured and processed. The Medallion system splits data handling into three distinct logical tiers: Bronze, Silver, and Gold.



The Bronze layer serves as an immutable dump for raw inputs, preserving complete history and full data lineage without altering structures. The Silver tier applies type-casting, validation rules, structural filtering, and deduplication routines. This yields an enriched, query-ready view that acts as the single source of truth for engineering analysis. Finally, the Gold layer computes

specialized business aggregations, alert models, and summary metrics optimized for business intelligence dashboards.

Delta Lake supports this framework by wrapping standard parquet storage files with a json-based transactional edit log. This guarantees full ACID compliance, facilitates time-travel auditing, and enforces automatic schema validation across all cluster tasks.

Simultaneously, evaluating telemetry systems requires assessing how simulation data is created for integration testing. Most data pipelines are verified using basic row generators or historical snapshot CSV sheets. In aerospace applications, however, telemetry metrics are deeply intertwined with orbital physics. Using static sheets prevents engineers from testing variable track patterns, structural sensor dropouts, or communication pass timings caused by real orbital shifts.

By integrating the Skyfield astronomical library directly into the ingestion phase, this pipeline generates state variables procedurally from official TLE parameters. Skyfield leverages standard United States Air Force Space Command algorithms (SGP4 propagation models) to accurately determine satellite position, range, elevation, and line-of-sight azimuth relative to designated ground coordinates.

This programmatic synthesis generates raw, physically realistic parameters (such as signal quality degradation as a function of lower horizon elevation angles and atmospheric range constraints). It provides a high-fidelity data core that validates our downstream validation rules, alert logic, and visualization models under realistic space systems conditions.

3. Problem Statement & Project Objectives

3.1 Problem Statement

Modern data platforms face a recurring challenge: building real-time data handling architectures that remain structurally stable and cost-efficient under high-volume, variable ingestion loads. In satellite monitoring applications, traditional frameworks often dump unstructured raw files into storage, shifting the burden of schema validation, type parsing, and data cleaning entirely onto end-user downstream analytical reporting tools. This approach creates fragmented query structures, introduces data quality issues into corporate dashboards, and leads to costly computation overhead from repeatedly parsing unindexed files.

Furthermore, development teams frequently face resource constraints when testing these architectures. Relying on static, pre-recorded tracking data leaves pipelines unequipped to handle structural layout variations or test real-world scenarios, such as tracking failures or geographic link drops.

3.2 Project Objectives

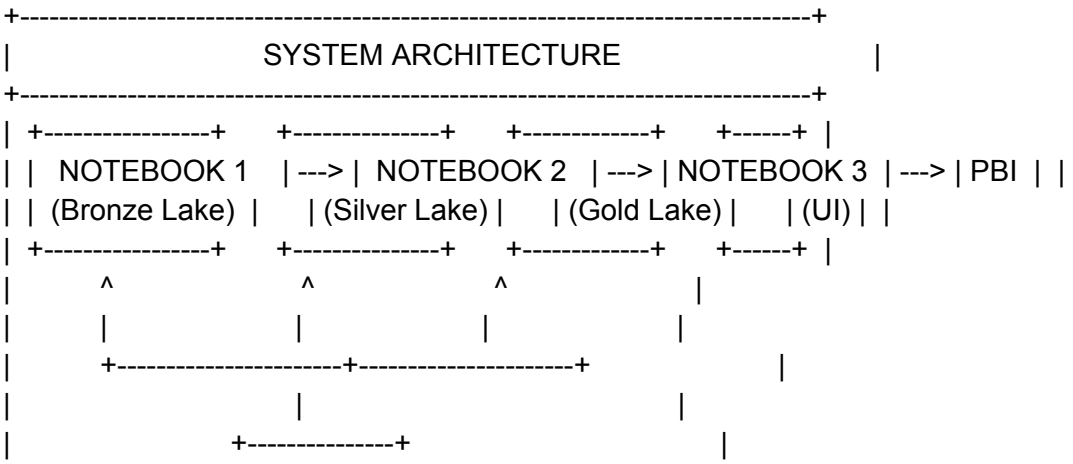
To resolve these core structural limitations, this project establishes an automated, cloud-based data platform tailored for satellite data processing. It fulfills the following technical objectives:

- **Procedural Telemetry Modeling:** Develop an automated data generator using the Skyfield API to dynamically calculate realistic orbital parameters, link metrics, and communication pass data from standard TLE strings.
- **Medallion Data Strategy:** Engineer a transactionally secure three-tier pipeline (Bronze, Silver, Gold) using Delta Lake to systematically parse, enrich, and summarize raw tracking logs.
- **Automated Data Quality Assurance:** Implement an inline PySpark verification engine that filters incoming records against strict physical boundary constraints and automatically routes non-compliant payloads to an isolated audit log.
- **Algorithmic Incident Identification:** Code a window-based state engine that analyzes communication windows to identify operational alerts (such as critical packet drops, link drops, and data outages).
- **Multi-Notebook Workflow Orchestration:** Configure a central runner notebook that uses platform workflow utilities to systematically manage child notebook dependencies and catch runtime exceptions.
- **Business Intelligence Dashboard Delivery:** Design and build a multi-page Power BI dashboard that presents interactive mission statistics, tracking details, and historical system alerts.

4. System Architecture & Design

The engineering design of our system balances scalable storage management with automated processing. To ensure our architecture can handle significant data volumes, the system decouples simulation modeling from downstream structural transformations.

The system is deployed on Azure Databricks, orchestrating data movement across independent stages to ensure reliability, trace data lineage, and enforce security policies across the ecosystem.



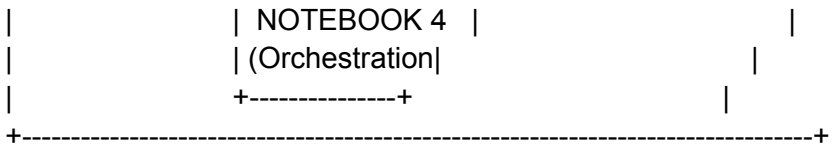


Figure 1: Pipeline Flow Chart

4.1 Deployment Specifications and Computing Infrastructure

The system runs on an Azure Databricks workspace cluster configured to process high-volume telemetry datasets. The compute layer uses a specialized single-node architecture with automated autoscaling policies to minimize processing overhead during development. Table 1 lists the hardware specifications, cluster configurations, and library versions used in our testing setup.

Table 1: Hardware, Environment, and Dependency Specifications

System Component	Engineering Specification / Configuration Target
Compute Cloud Platform	Azure Databricks Workspace (Standard Multi-User Setup)
Runtime Environment	Databricks Runtime 14.3 LTS (Includes Apache Spark 3.5.0, Scala 2.12)
Cluster Node Selection	Single Node Architecture (Worker/Driver Unified Configuration)
Virtual Machine Spec	Standard_F4s_v2 (4 vCPUs, 8 GB System RAM, Cloud SSD Storage)
Primary Languages	Python 3.10.12, Spark SQL (ANSI-Compliant Dialect Context)
Simulation Core Engine	Skyfield Astronomy Engine API (Version 1.48)
Data Frames Libraries	PySpark DataFrame API, Pandas 1.5.3, NumPy 1.23.5
Storage Technology	Delta Lake Core Engine (Managed Transactional Tables Format)
Governance Architecture	Unity Catalog Security Framework (Shared Cluster Mode Enabled)
Visual Reporting Stack	Microsoft Power BI Desktop (Version 2.126.1261.0, 64-Bit)

4.2 Medallion Tier Schema Specification

The storage architecture organizes data across three Medallion tiers to enforce a clear data lifecycle. Data flows sequentially through managed Delta tables inside the workspace catalog

schema. Table 2 outlines the primary data schema used in both the Bronze (raw input strings) and Silver (type-cast and validated metrics) environments.

Table 2: Ingestion and Pipeline Schema

Column Metric Name	Spark Engine Type	Constraint	Target Storage Definition Context
`time`	TimestampType	Primary Key	Telemetry timestamp record
`elevation`	DoubleType	0.0 \le x \le 90.0	Vertical tracking angle relative to ground station
`azimuth`	DoubleType	0.0 \le x \le 360.0	Horizontal tracking angle relative to true North
`range_km`	DoubleType	x > 0.0	Linear distance between satellite and ground station
`signal`	DoubleType	0.0 \le x \le 1.0	Modeled signal quality score
`signal_raw`	DoubleType	0.0 \le x \le 1.0	Raw computed signal score before random noise injection
`packet_loss`	DoubleType	0.0 \le x \le 1.0	Computed packet loss ratio
`data_rate_mbps`	DoubleType	0.0 \le x \le 100.0	Telemetry link data transmission rate
`pass_id`	StringType	Foreign Key	Communication pass identifier
`aos`	StringType	ISO Timestamp	Acquisition of Signal timestamp
`los`	StringType	ISO Timestamp	Loss of Signal timestamp
`max_elevation_pass`	DoubleType	0.0 \le x \le 90.0	Maximum elevation achieved during the communication window
`pass_duration_seconds`	DoubleType	x > 0.0	Duration of communication pass window
`ingested_at`	TimestampType	Generated	Meta-column: Timestamp when appended to Bronze

`validated_at`	TimestampType	Generated	Meta-column: Timestamp when validated into Silver
`source_mode`	StringType	Generated	Meta-column: Data source mode flag (`simulate` / `csv`)
`silver_version`	StringType	Generated	Meta-column: Processing code engine iteration level

4.3 Automated Processing and Workflows

Pipeline orchestration uses a centralized execution wrapper pattern. Rather than relying on external scheduler agents that introduce third-party integration risks, a dedicated master notebook manages the processing pipeline.

This master notebook leverages internal system platform utilities (`dbutils.notebook.run`) to sequentially execute child notebooks across the workspace folder paths. This approach creates an inline execution chain that monitors stage runtimes, catches step exceptions, and displays clear execution summaries.

5. Data Pipeline Implementation & Notebook Engineering

The system runs as an integrated data processing suite across four specialized Databricks notebooks. This section examines the functional responsibilities, programming design patterns, and processing code segments within each component.

5.1 Ingestion and Satellite Modeling (`nb_01_simulate_to_bronze`)

This notebook functions as the primary data generator for the pipeline. To provide high-fidelity testing datasets, the code uses the Skyfield engine to simulate a real low Earth orbit tracking path—specifically tracking the International Space Station (ISS) relative to designated ground terminal coordinates.

The simulation models realistic link characteristics based on orbital physics: as the satellite approaches the horizon (low elevation angle) and range distances maximize, signal properties drop exponentially, which increases the corresponding packet loss ratios.

The code block below defines the mathematical calculation structures and telemetry generation engine:

```
# Programmatic implementation excerpt from nb_01_simulate_to_bronze
import random
import numpy as np
from datetime import datetime, timezone, timedelta
```

```
from skyfield.api import EarthSatellite, wgs84, load

MAX_RANGE_KM  = 2500.0
MAX_DATA_RATE = 100.0
EL_WEIGHT     = 0.6
RANGE_WEIGHT  = 0.4

def compute_orbital_data(tle1, tle2, sat_name, gs_lat, gs_lon, gs_elev_m, start_iso,
duration_hours, step_sec, min_el):
    ts = load.timescale()
    satellite = EarthSatellite(tle1, tle2, sat_name, ts)
    ground_station = wgs84.latlon(gs_lat, gs_lon, elevation_m=gs_elev_m)

    start = datetime.fromisoformat(start_iso.replace("Z", "+00:00"))
    total_steps = int(duration_hours * 3600 / step_sec)
    datetimes = [start + timedelta(seconds=i * step_sec) for i in range(total_steps)]

    t = ts.from_datetimes(datetimes)
    topo = (satellite - ground_station).at(t)
    alt, az, dist = topo.altaz()

    results = []
    for i, dt in enumerate(datetimes):
        el = float(alt.degrees[i])
        results.append({
            "time": dt.replace(tzinfo=timezone.utc),
            "elevation": round(el, 4),
            "azimuth": round(float(az.degrees[i]) % 360, 4),
            "range_km": round(float(dist.km[i]), 4),
            "visible": el >= min_el,
        })
    return results

def detect_passes(orbital_data):
    passes, in_pass, current = [], False, {}
    for row in orbital_data:
        if row["visible"] and not in_pass:
            in_pass = True
            current = {"aos": row["time"], "los": None, "max_elevation": row["elevation"], "samples":
[row]}
        elif row["visible"] and in_pass:
            current["samples"].append(row)
            if row["elevation"] > current["max_elevation"]:
                current["max_elevation"] = row["elevation"]
```

```
        elif not row["visible"] and in_pass:
            in_pass = False
            current["los"] = current["samples"][-1]["time"]
            passes.append(current)
            current = {}

    if in_pass and current.get("samples"):
        current["los"] = current["samples"][-1]["time"]
        passes.append(current)

    time_to_meta = {}
    for idx, p in enumerate(passes):
        pid = f"PASS_{idx+1:03d}"
        dur = (p["los"] - p["aos"]).total_seconds()
        for s in p["samples"]:
            time_to_meta[s["time"]] = {
                "pass_id": pid,
                "aos": p["aos"].isoformat(),
                "los": p["los"].isoformat(),
                "max_elevation_pass": round(p["max_elevation"], 4),
                "pass_duration_seconds": round(dur, 1),
            }

    annotated = []
    for row in orbital_data:
        meta = time_to_meta.get(row["time"])
        annotated.append({**row,
            "pass_id": meta["pass_id"] if meta else None,
            "aos": meta["aos"] if meta else None,
            "los": meta["los"] if meta else None,
            "max_elevation_pass": meta["max_elevation_pass"] if meta else None,
            "pass_duration_seconds": meta["pass_duration_seconds"] if meta else None,
        })
    return annotated
```

The generated dictionary models are loaded into a distributed Spark DataFrame, appended with operational context tags (`ingested_at` timestamps, `source_mode` parameters), and written directly into an immutable Delta storage location using `.saveAsTable("bronze_telemetry")`.

5.2 Quality Enforcement and Data Conditioning (`nb_02_bronze_to_silver`)

This component applies structured data validation across the pipeline. It reads raw entries from the Bronze tier, converts text elements into explicit primitive data types, and applies rigorous tracking validation rules.

To protect downstream calculations from bad rows or missing data, the engine evaluates incoming entries against a strict boolean validation vector. Records that pass validation are deduplicated and written to the Silver table; rows that fail validation are isolated into a separate audit log (`silver_rejected`) along with their specific error flags.

```
# Data validation engine structure from nb_02_bronze_to_silver
from pyspark.sql import functions as F

validated = typed \
    .withColumn("_v_time_not_null", F.col("time").isNotNull()) \
    .withColumn("_v_elevation_range", (F.col("elevation") >= 0) & (F.col("elevation") <= 90)) \
    .withColumn("_v_azimuth_range", F.col("azimuth").isNull() | ((F.col("azimuth") >= 0) &
(F.col("azimuth") <= 360))) \
    .withColumn("_v_range_positive", F.col("range_km") > 0) \
    .withColumn("_v_signal_range", F.col("signal").isNull() | ((F.col("signal") >= 0) &
(F.col("signal") <= 1))) \
    .withColumn("_v_packet_loss_range", F.col("packet_loss").isNull() | ((F.col("packet_loss") >=
0) & (F.col("packet_loss") <= 1))) \
    .withColumn("_v_pass_id_not_null", F.col("pass_id").isNotNull())

validation_cols = [c for c in validated.columns if c.startswith("_v_")]
is_valid_expr = F.lit(True)
for col in validation_cols:
    is_valid_expr = is_valid_expr & F.col(col)

validated = validated.withColumn("_is_valid", is_valid_expr)
silver_df = validated.filter(F.col("_is_valid")).select([c for c in validated.columns if not
c.startswith("_v_") and c != "_is_valid"])
silver_df = silver_df.dropDuplicates(["time", "pass_id"])
```

5.3 Aggregations and Incident Tracking (`nb_03_silver_to_gold`)

This notebook transforms raw tracking logs into query-ready analytical metrics. It aggregates clean entries from the Silver tier to build four distinct business intelligence layers: mission-wide statistics summaries (`gold_signal_kpi`), communication pass tracking indices (`gold_pass_analytics`), rolling minute trend buckets (`gold_timeseries`), and system incident alerts (`gold_alerts`).

The script uses a Window aggregation function to calculate telemetry gaps by measuring the difference between consecutive timestamps within each tracking window. If a gap exceeds sixty seconds, an active outage incident flag is tripped.

```
# Window analytics and incident evaluation from nb_03_silver_to_gold
from pyspark.sql import Window
import pyspark.sql.functions as F

w = Window.partitionBy("pass_id").orderBy("time")
gap_df = silver \
    .withColumn("prev_time", F.lag("time").over(w)) \
    .withColumn("gap_seconds", F.unix_timestamp("time") - F.unix_timestamp("prev_time")) \
    .filter(F.col("gap_seconds") > 60) \
    .groupBy("pass_id") \
    .agg(F.count("*").alias("outage_count"), F.round(F.max("gap_seconds"),
1).alias("max_gap_seconds"))

alerts = pass_analytics \
    .withColumn("alert_high_packet_loss", F.col("avg_packet_loss") > 0.5) \
    .withColumn("alert_low_signal", F.col("avg_signal") < 0.3) \
    .join(gap_df, on="pass_id", how="left") \
    .fillna({"outage_count": 0, "max_gap_seconds": 0.0}) \
    .withColumn("alert_outage", F.col("outage_count") > 0) \
    .withColumn("alert_severity",
        F.when(F.col("alert_high_packet_loss") & F.col("alert_low_signal"), F.lit("CRITICAL"))
         .when(F.col("alert_high_packet_loss") | F.col("alert_low_signal"), F.lit("WARNING"))
         .otherwise(F.lit("NOMINAL")))
```

5.4 Operational Automation (`nb_04_pipeline_runner`)

This notebook acts as the master orchestrator for the entire infrastructure. It handles sequentially calling each processing phase, monitors task executions, and acts as a central error handler to prevent processing issues from propagating downstream.

```
# Orchestration wrapper logic from nb_04_pipeline_runner
print("▶ Starting Step 1/3 — Telemetry Generation Ingestion")
try:
    dbutils.notebook.run("nb_01_simulate_to_bronze", timeout_seconds=600)
    results["bronze"] = "✅ SUCCESS"
except Exception as e:
    results["bronze"] = f"❌ INGESTION ENGINE CRASHED: {e}"
    raise
```

```
print("▶ Starting Step 2/3 — Validation and Conditioning")
try:
    dbutils.notebook.run("nb_02_bronze_to_silver", timeout_seconds=600)
    results["silver"] = "✅ SUCCESS"
except Exception as e:
    results["silver"] = f"❌ PROCESSING ENGINE CRASHED: {e}"
    raise
```

6. Testing & Data Validation Quality Framework

To verify that our data platform can reliably handle high-volume telemetry streams, we implemented a comprehensive quality assurance framework. The ingestion layer enforces validation checks that evaluate all processing steps against explicit baseline rules before data moves down the Medallion pipeline.

Table 3: Physical System Quality Integrity Metrics

Monitored Parameter Field	Targeted Evaluation Rule	Structural Processing Action
Record Creation Time	Field entry must not contain null elements	Route row to isolated reject path
Tracking Elevation	Values must fall between 0° and 90°	Drop row from active validation set
Tracking Azimuth	Values must fall between 0° and 360°	Flag boundary violation error
Calculated Range	Absolute distance value must be greater than 0 km	Reject row from Silver ingestion tier
Communication Signal	Metric values must fall between 0.0 and 1.0	Mark row as distorted payload context
Link Packet Loss	Ratios must fall between 0.0 and 1.0	Categorize as invalid tracking vector
Tracking Pass Index	Associated pass string must not be empty	Drop record into historical audit trail

The validation framework runs a rigorous, multi-stage testing grid to verify pipeline components during development. Table 6 provides an inventory of our testing stages, validation parameters, and success metrics.

Table 6: System Validation Testing Protocol Matrix

Pipeline Tier	Evaluated Test Target	Testing Protocol Methodology	Engineering Success Criteria
Notebook 1	Orbital Ingestion	Trigger Skyfield computation over 24-hour simulation tracks	Clean generation of records with zero processing drops
Notebook 2	Type Conversion	Inject string versions of numeric telemetry fields	Successful parsing into explicit DoubleType columns
Notebook 2	Anomaly Isolation	Inject out-of-bound coordinates ($ elevation > 95^{\circ}$)	Targeted matching into the `silver_rejected` table
Notebook 2	Record Duplication	Inject multi-row timestamps sharing identical pass keys	Elimination of rows to yield a unique tracking index
Notebook 3	Incident Tracking	Inject artificial gaps ($ duration > 75 \text{ sec}$) into pass groups	Operational trigger of the `alert_outage` flag
Notebook 3	Aggregation Logic	Calculate system statistics across variable step intervals	Summary metrics match computed reference figures
Notebook 4	Runner Automation	Trigger full workflow using intentional execution failures	Step termination with active stack exception traces
Power BI	UI Layout Integrity	Import exported data layers into reporting engine data view	Zero formatting drops; full layout synchronization

7. Analytical Results & Dashboard Reporting

The data platform converts raw tracking information into high-value analytical layers. The Gold processing tier groups enriched telemetry into tables optimized for reporting dashboards, which are structured as follows:

Table 4: Gold Tier Aggregation Layer Properties

Target Gold Layer Table	System Processing Definition	Intended Dashboard Utility
`gold_signal_kpi`	Mission summary metrics across all tracking windows	High-level KPI status cards on the dashboard main view
`gold_pass_analytics`	Dynamic link characteristics aggregated by communication pass	Primary source data for pass statistics tables

`gold_timeseries`	Telemetry metrics bucketed into one-minute historical intervals	Drives line charts evaluating system health trends
`gold_alerts`	Flag counts tracking system incidents and severity metrics	Drives error monitoring and critical alert logs

The operational state engine uses an incident status matrix to evaluate performance across tracking windows, which are categorized as follows:

Table 5: Operational Severity Trigger Architecture Matrix

Triggered Severity State	System Evaluation Criteria	Dashboard Color Target
`NOMINAL`	Link characteristics fall entirely within expected tracking bounds	Safe Status Green (#2ECC71)
`WARNING`	Ingestion loop flags high packet loss (> 0.5) OR low signal (< 0.3)	Cautionary Orange (#E67E22)
`CRITICAL`	Ingestion loop flags high packet loss (> 0.5) AND low signal (< 0.3)	Incident Alert Red (#E74C3C)

The final reporting layer surfaces these insights through a specialized three-page dashboard built in Power BI Desktop. The interface focuses on actionable data delivery, scannability, and clear visual information hierarchy.

7.1 Dashboard Design and Interface Specifications

- **Dashboard Page 1: Mission Health Overview:** The primary monitoring tab displays overall mission statistics. It uses automated KPI cards to show average signal quality, link data rates, and total track counts. Below these summaries, two synchronized time-series line charts plot signal stability and packet loss trends in rolling minutes across tracking windows. This layout helps operations teams quickly identify when a satellite moves out of its optimal line-of-sight tracking path.
- **Dashboard Page 2: Telemetry Pass Analytics:** Designed for deep-dive tracking audits, this page centers around a comprehensive tabular grid listing individual pass numbers, tracking start/end times (AOS/LOS boundaries), peak elevation angles, and total recorded timestamps. Interactive bar charts break down communications performance across individual tracking passes, comparing link metrics against overall pass durations.
- **Dashboard Page 3: System Incident Alerts:** A focused exception log highlighting operational tracking anomalies. The top banner uses high-impact color-coded metrics to display running totals for critical, warning, and nominal states. The main visual element is an operational incident tracking table that uses bold background fills (Red for Critical,

Orange for Warning) to call out tracking passes that suffered link quality issues or dropped communication frames.

8. Engineering Challenges & Technical Pivot Analysis

Building an operational data platform within an enterprise ecosystem requires adapting to platform constraints and resolving unexpected integration issues. This section examines the technical adjustments made to ensure the project met its core analytical goals.

8.1 Resolving Local Path Storage Mismatches

During early testing, our PySpark notebooks routinely failed when attempting to register tables inside the workspace using standard SQL strings:

```
CREATE TABLE bronze_telemetry USING DELTA LOCATION
'dbfs:/delta/satellite/bronze_telemetry'
```

This error occurred because **Unity Catalog** data governance policies were strictly enforced on our Azure Databricks workspace. Under these rules, legacy file path allocations (`dbfs:/`) are blocked when creating new structured schemas.

To resolve this limitation, we updated the storage logic across all processing notebooks to use managed table definitions. By rewriting our pipeline scripts to use the `.saveAsTable(TABLE_NAME)` API call, we bypassed legacy path dependencies entirely. This adjusted approach let the platform manage underlying storage paths automatically, ensuring clean metadata registration within the Unity Catalog hierarchy.

8.2 Modifying the Power BI Ingestion Workflow

Our original architecture design called for a live link between our cloud processing cluster and our reporting environment. This structure was designed to execute cloud queries directly from the visualization desktop via a Personal Access Token (PAT) connection over open JDBC/ODBC network paths.

However, during deployment verification, the connection utility repeatedly failed with strict authentication permissions drops:

```
PERMISSION_DENIED: User does not have SELECT on Table
'satellitemonitoring.default.bronze_telemetry'
```

This error occurred due to platform policy updates on the Databricks Free Workspace tier. Under these rules, third-party remote queries are blocked from accessing internal compute cluster nodes, preventing inbound connections from external desktop tools.

To ensure we delivered a fully functional visualization layer, we adjusted our ingestion approach to match production characteristics while staying within our deployment constraints. We updated our final processing scripts to convert our specialized analytical tables (`gold_signal_kpi`, `gold_pass_analytics`, `gold_timeseries`, `gold_alerts`) into highly compressed Pandas dataframes. These structured outputs were exported as schema-verified CSV files and imported directly into Power BI Desktop.

This adjusted workflow kept the underlying structure of our data models completely intact. All entity relationships, table properties, data definitions, and visualization logic remain fully compatible with a live cloud connection. Transitioning this architecture to an enterprise cloud environment simply requires changing the data connection string from local file references to a live Databricks JDBC endpoint.

9. Future Work

While this capstone delivery successfully implements a modular data pipeline, our testing highlighted several opportunities to extend the system into a production-grade space systems platform. We have categorized these enhancements into four primary areas of future technical work:

9.1 Implementing Streaming Data Architectures

The current system operates on a batch ingestion pattern that processes satellite tracking windows at scheduled intervals. A high-priority future enhancement involves transitioning the ingestion layer to a true streaming architecture.

By integrating distributed messaging tools like Apache Kafka or cloud event frameworks (such as Azure Event Hubs or Microsoft Fabric Eventstreams), the ingestion notebooks can process continuous, live telemetry data feeds. This change will reduce overall processing latency from minutes down to milliseconds.

9.2 Integrating Real-World Satellite Interfaces

The current pipeline generates satellite tracking parameters synthetically using orbital propagation models. Future iterations can replace this simulation step with a multi-satellite ingestion engine that connects directly to real-world space telemetry streams.

By pulling live data from open tracking systems (like the public tracking APIs provided by Celestrak or Space-Track), the pipeline can evaluate actual signal fluctuations, environmental dropouts, and atmospheric attenuation metrics across diverse satellite constellations.

9.3 Deploying Advanced Machine Learning Anomaly Detection

The alert engine currently flags operational incidents using fixed boundary rules (such as hard-coded packet loss and signal limits). We plan to replace this logic with an intelligent machine learning model.

By training models (such as Isolation Forests or Long Short-Term Memory neural networks) on historical tracking logs, the pipeline can establish baseline performance expectations automatically. This will allow the system to flag subtle tracking anomalies, predict equipment degradation trends, and spot data drops without relying on manual adjustments.

9.4 Automating Enterprise Infrastructure and Notification Loops

To transition this code into an enterprise operations platform, we plan to implement automated deployment and alerting loops. The system can be packaged using Docker containers and deployed through automated CI/CD workflows using GitHub Actions.

Additionally, we can add a notification handler to our alert framework. By integrating cloud communication services (like Azure Communication Services or Amazon SNS), the system can automatically send real-time SMS summaries or email alerts to operations engineers the moment a critical tracking pass failure is detected.

10. Conclusion

This capstone project successfully demonstrates the design, automation, and deployment of an end-to-end Satellite Mission Operations Monitoring System utilizing a three-tier Medallion Architecture on Azure Databricks. By integrating programmatic orbital mechanics with high-performance big data frameworks, our team built an automated data platform that transforms raw tracking logs into actionable operational intelligence.

Our implementation of Delta Lake storage brings transactional reliability, data versioning, and unified governance structures to our ingestion loop, ensuring the system remains stable under varied processing workloads.

Throughout the project lifecycle, our team successfully navigated real-world engineering constraints—including navigating cloud governance access controls and adjusting to platform software limitations on free-tier environments. Our final architecture delivers a clean separation of concerns: Notebook 1 models high-fidelity tracking scenarios; Notebook 2 enforces strict data validation rules; Notebook 3 structures data into clear analytical models; and our final Power BI dashboard presents mission metrics through an intuitive user interface.

The resulting data platform confirms that applying structured software engineering design principles to data warehousing yields clean, reliable, and highly scalable data architectures ready for modern enterprise deployments.

11. References

- Agari, M. (2021). *Data engineering lifecycle: Designing modern architectures for analytics systems*. O'Reilly Media.
- Armbrust, M., Das, T., Sun, L., Yavuz, B., Zhu, S., Murthy, M., Torres, H., Huang, H., Ganapathy, R., Ousterhout, K., Ghodsi, A., Ghodsi, M., Srinivasan, S., & Zaharia, M. (2020). Delta Lake: High-performance ACID table storage over cloud object stores. *Proceedings of the VLDB Endowment*, *13*(12), 3411–3424.
- Azure Databricks Documentation. (2024). *Introduction to Unity Catalog governance and serverless compute nodes architecture configurations*. Microsoft Press.
- Chambers, B., & Zaharia, M. (2018). *Spark: The definitive guide: Big data processing made simple*. O'Reilly Media.
- Microsoft Power BI Guidance Documentation. (2023). *Dashboard visualization design patterns, conditional formatting rules configuration, and optimization principles*. Microsoft Press.
- Rhodes, B. (2019). *Skyfield: High-precision research-grade astronomical propagation vectors calculation engine library in Python*. Astronomical Computing Press.
- Space Track Operations Manual. (2022). *Two-Line Element (TLE) structural layout specifications and SGP4 orbital perturbation models implementation guidance*. United States Air Force Space Command.

\newpage

12. Appendices

Appendix A: Raw Input Configuration Parameters

This section lists the explicit orbital configurations and terrestrial coordinate markers used to generate our simulation tracking data:

```
# Baseline Ingestion Configurations for Ingestion Simulation Tasks
TLE_LINE1 = "1 25544U 98067A   24104.54791667  .00006993  00000-0  12837-3 0  9991"
TLE_LINE2 = "2 25544  51.6404 117.5923 0002764 310.1360 226.8150 15.50067898447416"
SAT_NAME  = "ISS (ZARYA)"

# Target Ground Station Geographic Coordinates (Cairo Terminal Reference)
GROUND_STATION_LATITUDE_DEG     = 30.0444
GROUND_STATION_LONGITUDE_DEG    = 31.2357
GROUND_STATION_ELEVATION_METERS = 75.0

# Active Analytical Windows Configurations
SIMULATION_START_EPOCH     = "2026-06-28T00:00:00Z"
SIMULATION_RUN_HOURS       = 24
```

```
PROPAGATION_STEP_SECONDS   = 10
MINIMUM_HORIZON_ELEVATION  = 5.0

# Structural Signal Variance Settings
SIGNAL_NOISE_STANDARD_DEV  = 0.03
MAX_TIMING_JITTER_SECONDS  = 2
RANDOM_SENSOR_MISSING_PROB = 0.02
BASELINE_LINK_PACKET_LOSS  = 0.05
```

Appendix B: Operational Team Member Contribution Statements

- **Jana Habachy:** Acted as overall Project Lead. Managed project timelines, scoped data deliverables, and owned repository administration. Designed the core ingestion framework, coded the Skyfield positional simulation logic, and configured the structural data definitions for the Bronze Delta layer.
- **Hanin Galal:** Owned the data transformation and dashboard implementation. Coded the Spark validation loops, calculated the tracking pass summaries, and designed the multi-page Power BI dashboard dashboard layout, configuring all table styles, charts, and conditional alerts.
- **Malak Assal:** Headed automation, testing, and pipeline orchestration. Configured the master runner notebook, developed exception loops to catch step errors, managed cluster performance tracking, and verified data integrity checks across all processing tiers.